

Programación Orientada a Objetos

Actividad 1: Individual (Valor 10%)

Estudiante:	Kevin Santiago Montoya Chavarria
Docente:	Walter Hugo Arboleda Mazo
Asignatura:	Programación Orientada a Objetos (2026-2S)
Lenguaje de Desarrollo:	Python (Paradigma Orientado a Objetos)
Fecha de Entrega:	Jueves 17 de Septiembre
Correo de Envío:	ia.walterarboleda@gmail.com

Tabla de Contenido

Sección	Descripción	Página
Punto 1	Ejercicio Resuelto No 4 (Cálculo de Edades)	Página 2
Punto 2	Ejercicio Resuelto No 5 (Prueba de Escritorio / Seguimiento)	Página 3
Punto 3	Ejercicio Propuesto No 12 (Cálculo de Salario y Retención)	Página 4
Punto 4	Ejercicio Propuesto No 14 (Cuadrado y Cubo de un Número)	Página 5
Punto 5	Ejercicio Propuesto No 17 (Área y Perímetro de Circunferencia)	Página 6

URL del Repositorio en GitHub:
<https://github.com/Kevin007ch/POO-Kevin-Santiago-Montoya-Chavarria>

Ejercicio Resuelto No 4

Enunciado: A la mamá de Juan le preguntan su edad, y contesta: tengo 3 hijos, pregúntele a Juan su edad. Alberto tiene $\frac{2}{3}$ de la edad de Juan, Ana tiene $\frac{4}{3}$ de la edad de Juan y mi edad es la suma de las tres. Hacer un algoritmo que muestre la edad de los cuatro.

Código Fuente en Python (POO)

```
class Edades:
    def __init__(self, edad_juan: float):
        self.edad_juan = edad_juan
        self.edad_alberto = self.calcular_edad_alberto()
        self.edad_ana = self.calcular_edad_ana()
        self.edad_mama = self.calcular_edad_mama()

    def calcular_edad_alberto() -> float:
        return self.edad_juan * (2 / 3)

    def calcular_edad_ana(self) -> float:
        return self.edad_juan * (4 / 3)

    def calcular_edad_mama(self) -> float:
        return self.edad_juan + self.edad_alberto + self.edad_ana

    def mostrar_edades(self) -> None:
        print(f"La edad de Juan es: {self.edad_juan}")
        print(f"La edad de Ana es: {self.edad_ana:.2f}")
        print(f"La edad de Alberto es: {self.edad_alberto:.2f}")
        print(f"La edad de la Mamá es: {self.edad_mama:.2f}")

if __name__ == "__main__":
    edad_juan = float(input("Ingrese la edad de Juan: "))
    familia = Edades(edad_juan)
    familia.mostrar_edades()
```

Diagrama de Clases UML

```
+-----+
| Edades |
+-----+
| - edad_juan: float |
| - edad_alberto: float |
| - edad_ana: float |
| - edad_mama: float |
+-----+
| + __init__(edad_juan: float) |
| + calcular_edad_alberto(): float |
| + calcular_edad_ana(): float |
| + calcular_edad_mama(): float |
| + mostrar_edades(): None |
+-----+
```

Ejercicio Resuelto No 5

Enunciado: Hacer un seguimiento (prueba de escritorio) del siguiente grupo de instrucciones:
INICIO | SUMA=0 | X=20 | SUMA=SUMA+X | Y=40 | X=X+Y**2 | SUMA=SUMA+X/Y | ESCRIBA:
"EL VALOR DE LA SUMA ES:", SUMA | FIN_INICIO

Prueba de Escritorio (Seguimiento Paso a Paso)

Paso / Instrucción	X	Y	SUMA	Operación / Observaciones
SUMA = 0	-	-	0	Inicialización de SUMA
X = 20	20	-	0	Asignación inicial de X
SUMA = SUMA + X	20	-	20	0 + 20 = 20
Y = 40	20	40	20	Asignación inicial de Y
X = X + Y**2	1620	40	20	20 + (40 ²) = 20 + 1600 = 1620
SUMA = SUMA + X/Y	1620	40	60.5	20 + (1620 / 40) = 20 + 40.5 = 60.5

Código Fuente en Python (POO)

```
import math

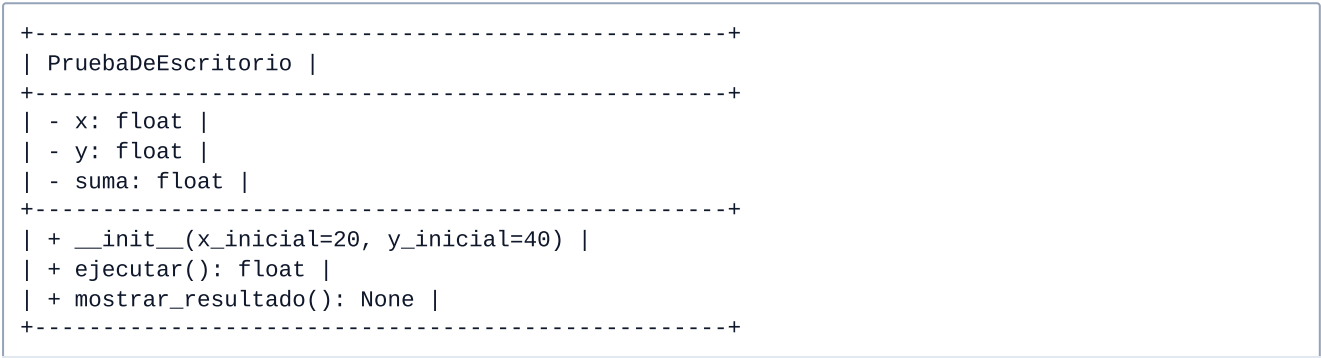
class PruebaDeEscritorio:
    def __init__(self, x_inicial: float = 20, y_inicial: float = 40):
        self.x = x_inicial
        self.y = y_inicial
        self.suma = 0.0

    def ejecutar(self) -> float:
        self.suma += self.x
        self.x += math.pow(self.y, 2)
        self.suma += (self.x / self.y)
        return self.suma

    def mostrar_resultado(self) -> None:
        resultado = self.ejecutar()
        print(f"El valor de la suma es: {resultado}")

if __name__ == "__main__":
    prueba = PruebaDeEscritorio()
    prueba.mostrar_resultado()
```

Diagrama de Clases UML



Ejercicio Propuesto No 12

Enunciado: Un empleado trabaja 48 horas en la semana a razón de \$5.000 hora. El porcentaje de retención en la fuente es del 12,5% del salario bruto. Se desea saber cuál es el salario bruto, la retención en la fuente y el salario neto del trabajador.

Código Fuente en Python (POO)

```
class Salario:
    def __init__(self, horas: float, valhora: float, porcentaje_retencion: float = 0.125):
        self.horas = horas
        self.valhora = valhora
        self.porcentaje_retencion = porcentaje_retencion
        self.salario_bruto = self.calcular_salario_bruto()
        self.retencion = self.calcular_retencion()
        self.salario_netto = self.calcular_salario_netto()

    def calcular_salario_bruto(self) -> float:
        return float(self.horas * self.valhora)

    def calcular_retencion(self) -> float:
        return self.salario_bruto * self.porcentaje_retencion

    def calcular_salario_netto(self) -> float:
        return self.salario_bruto - self.retencion

    def mostrar_resultados(self) -> None:
        print(f"El salario bruto es: ${self.salario_bruto:,.2f}")
        print(f"El valor de retención es: ${self.retencion:,.2f}")
        print(f"El salario neto es: ${self.salario_netto:,.2f}")

if __name__ == "__main__":
    empleado = Salario(48, 5000)
    empleado.mostrar_resultados()
```

Diagrama de Clases UML

```
+-----+
| Salario |
+-----+
| - horas: float |
| - valhora: float |
| - porcentaje_retencion: float |
| - salario_bruto: float |
| - retencion: float |
| - salario_netto: float |
+-----+
| + __init__(horas, valhora, pct_ret=0.125) |
| + calcular_salario_bruto(): float |
| + calcular_retencion(): float |
| + calcular_salario_netto(): float |
| + mostrar_resultados(): None |
+-----+
```

Ejercicio Propuesto No 14

Enunciado: Elabore un algoritmo que lea un número y obtenga su cuadrado y su cubo.

Código Fuente en Python (POO)

```
import math

class Numero:
    def __init__(self, numero: float):
        self.numero = numero
        self.cuadrado = self.calcular_cuadrado()
        self.cubo = self.calcular_cubo()

    def calcular_cuadrado(self) -> float:
        return math.pow(self.numero, 2)

    def calcular_cubo(self) -> float:
        return math.pow(self.numero, 3)

    def mostrar_resultados(self) -> None:
        print(f"El número ingresado es: {self.numero}")
        print(f"El cuadrado del número es: {self.cuadrado}")
        print(f"El cubo del número es: {self.cubo}")

if __name__ == "__main__":
    num = float(input("Ingrese un número: "))
    numero_obj = Numero(num)
    numero_obj.mostrar_resultados()
```

Diagrama de Clases UML

```
+-----+
| Numero |
+-----+
| - numero: float |
| - cuadrado: float |
| - cubo: float |
+-----+
| + __init__(numero: float) |
| + calcular_cuadrado(): float |
| + calcular_cubo(): float |
| + mostrar_resultados(): None |
+-----+
```

Ejercicio Propuesto No 17

Enunciado: Dado el radio de un círculo. Haga un algoritmo que obtenga el área del círculo y la longitud de la circunferencia.

Código Fuente en Python (POO)

```
import math

class Circulo:
    def __init__(self, radio: float):
        self.radio = radio
        self.area = self.calcular_area()
        self.perimetro = self.calcular_perimetro()

    def calcular_area(self) -> float:
        return math.pi * math.pow(self.radio, 2)

    def calcular_perimetro(self) -> float:
        return 2 * math.pi * self.radio

    def mostrar_resultados(self) -> None:
        print(f"El área del círculo es: {self.area:.2f}")
        print(f"El perímetro del círculo es: {self.perimetro:.2f}")

if __name__ == "__main__":
    radio = float(input("Ingrese el radio del círculo: "))
    circulo = Circulo(radio)
    circulo.mostrar_resultados()
```

Diagrama de Clases UML

```
+-----+
| Circulo |
+-----+
| - radio: float |
| - area: float |
| - perimetro: float |
+-----+
| + __init__(radio: float) |
| + calcular_area(): float |
| + calcular_perimetro(): float |
| + mostrar_resultados(): None |
+-----+
```

URL del Repositorio Oficial en GitHub:

<https://github.com/Kevin007ch/POO-Kevin-Santiago-Montoya-Chavarria>